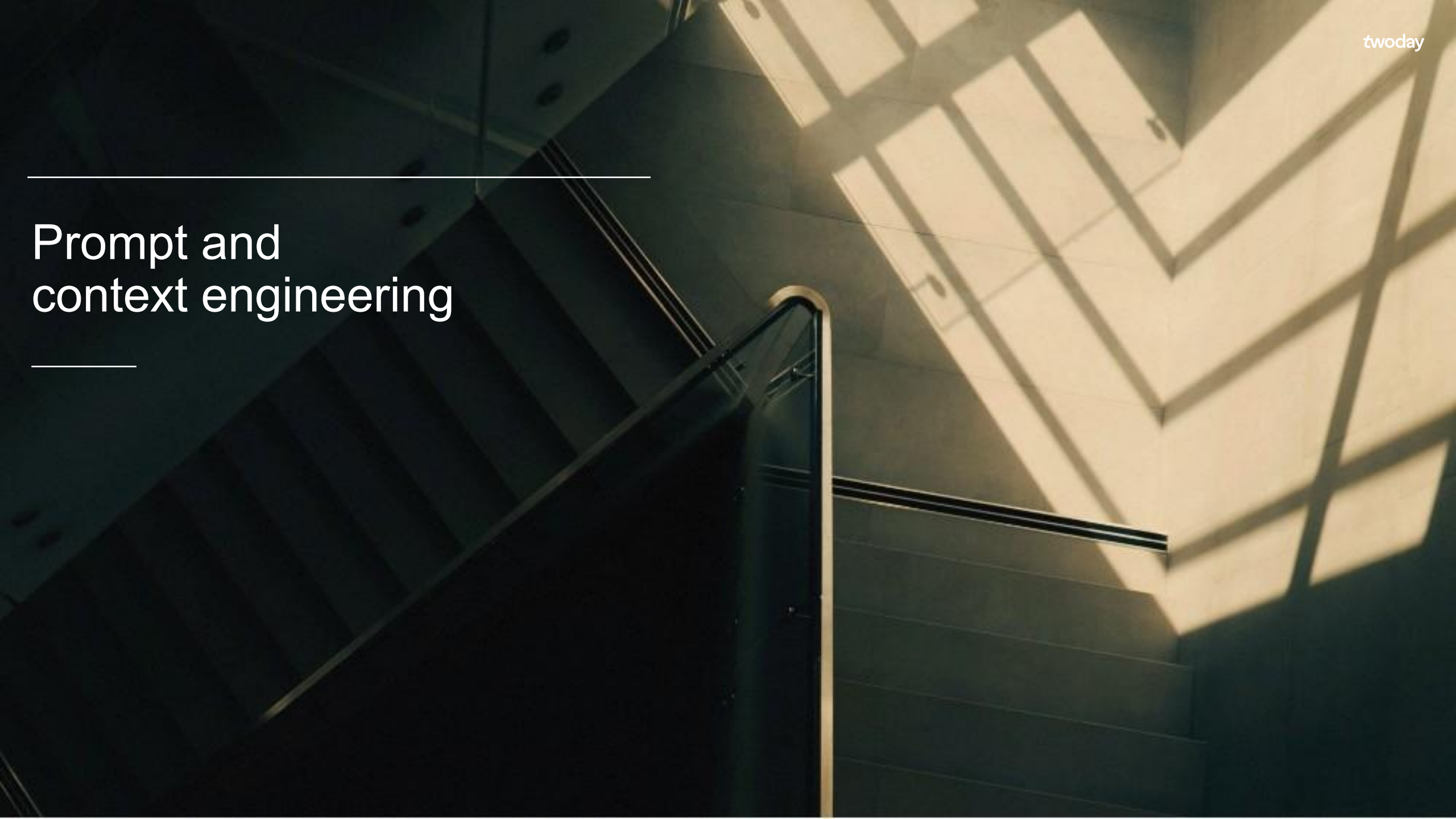
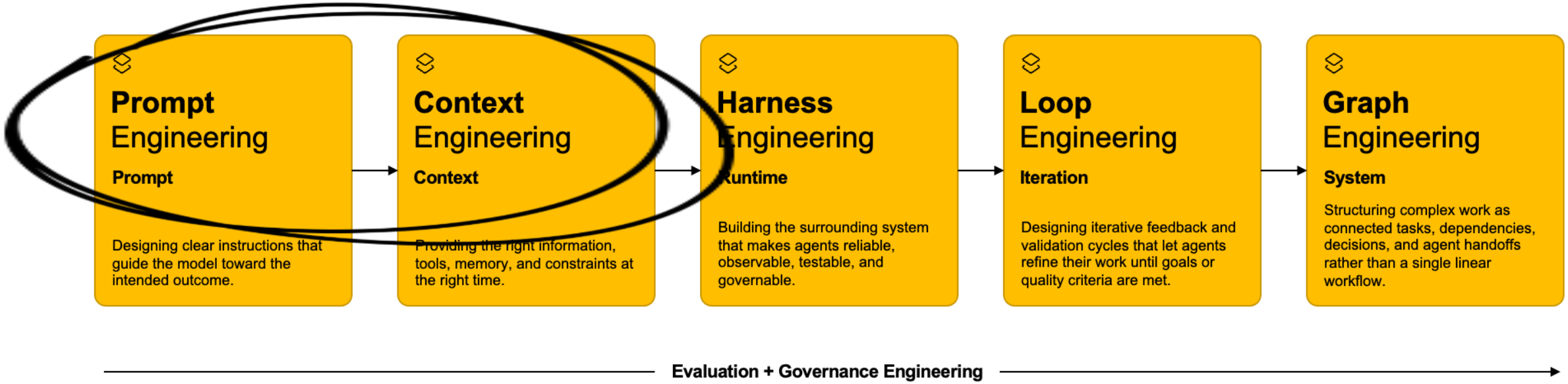


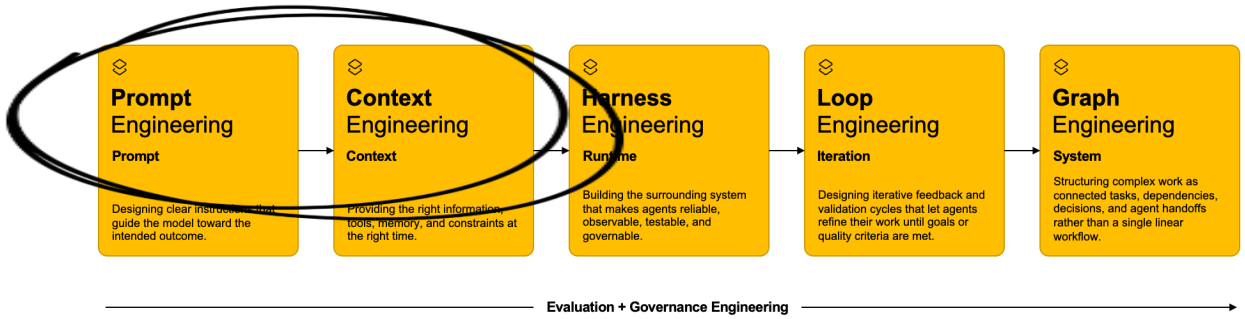
twoday

Prompt and context engineering


Prompt Engineering
Prompt
Designing clear instructions that guide the model toward the intended outcome.
Context Engineering
Context
Providing the right information, tools, memory, and constraints at the right time.
Harness Engineering
Runtime
Building the surrounding system that makes agents reliable, observable, testable, and governable.
Loop Engineering
Iteration
Designing iterative feedback and validation cycles that let agents refine their work until goals or quality criteria are met.
Graph Engineering
System
Structuring complex work as connected tasks, dependencies, decisions, and agent handoffs rather than a single linear workflow.
Evaluation + Governance Engineering

Two disciplines, one task

Closely related, but they solve different problems. Both are essential for effective agentic engineering, particularly when using agents such as Codex or GitHub Copilot to work on real software projects.



PROMPT ENGINEERING

Define the current task

Prompt engineering is about how you instruct an agent to perform a task.

Prompt engineering is **how you describe** the assignment: *"Implement the ability for users to cancel a submitted request. Ensure that requests already approved cannot be cancelled."*

CONTEXT ENGINEERING

Supply what the agent needs

Context engineering is about what information, tools, and constraints the agent has available when performing that task.

Context engineering is everything the engineer needs to **understand and implement** that assignment correctly:

- The application's architecture and existing code.
- The domain model and business rules.
- The team's coding conventions and technical standards.
- The available APIs, services, and dependencies.
- Existing tests and examples of similar functionality.
- The definition of done.

A useful prompt typically establishes four things



Engineering the instructions

A weak prompt

Add cancellation to the request system.

An engineered prompt

Implement cancellation for submitted requests. Users may cancel their own requests only while the status is Pending.

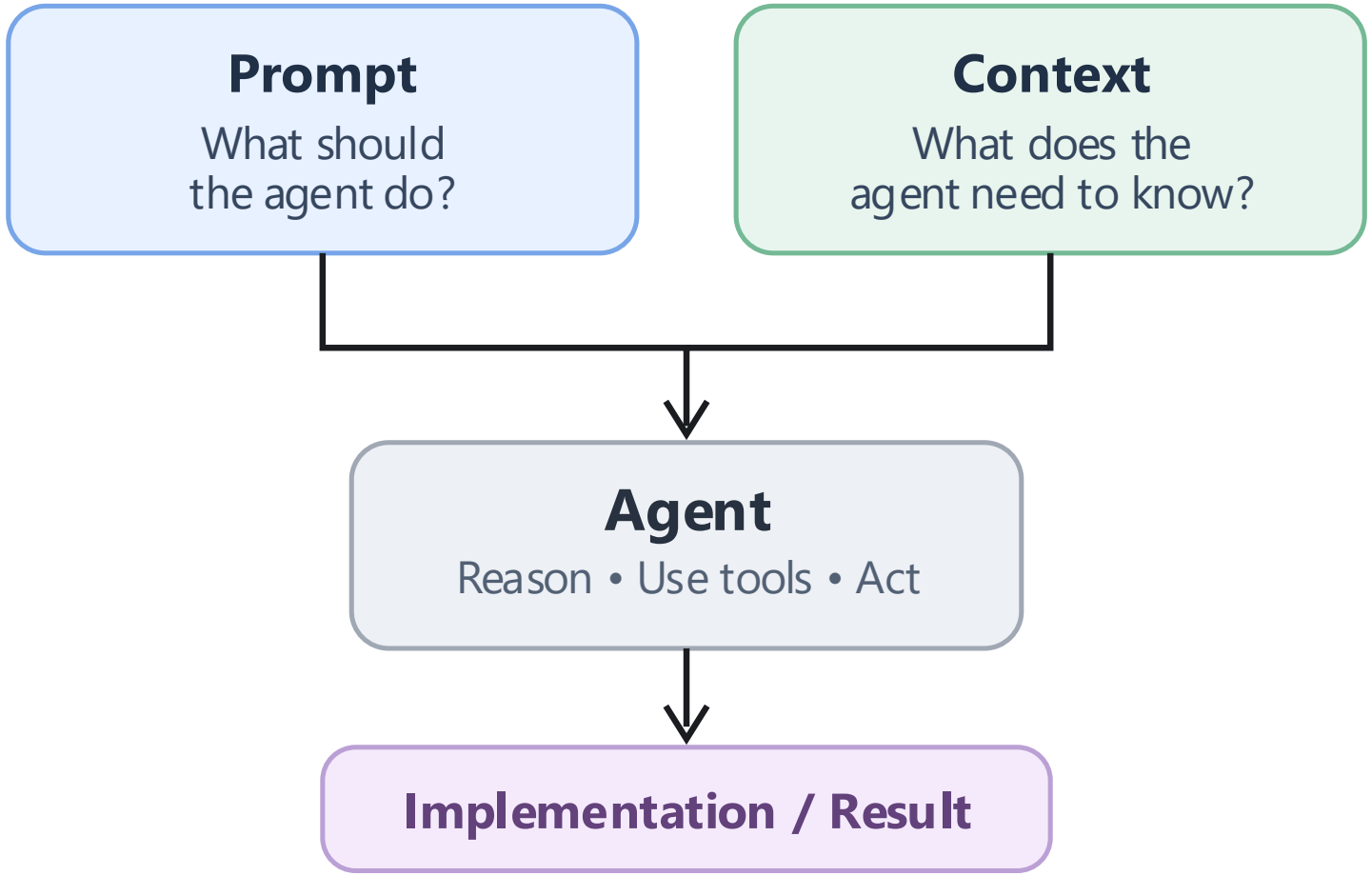
The operation must be handled through the existing application service and API patterns.

Do not introduce new dependencies or change unrelated functionality.

Add tests covering successful cancellation and attempts to cancel requests in other states.

Run the relevant tests and report the results.

How prompt and context work together

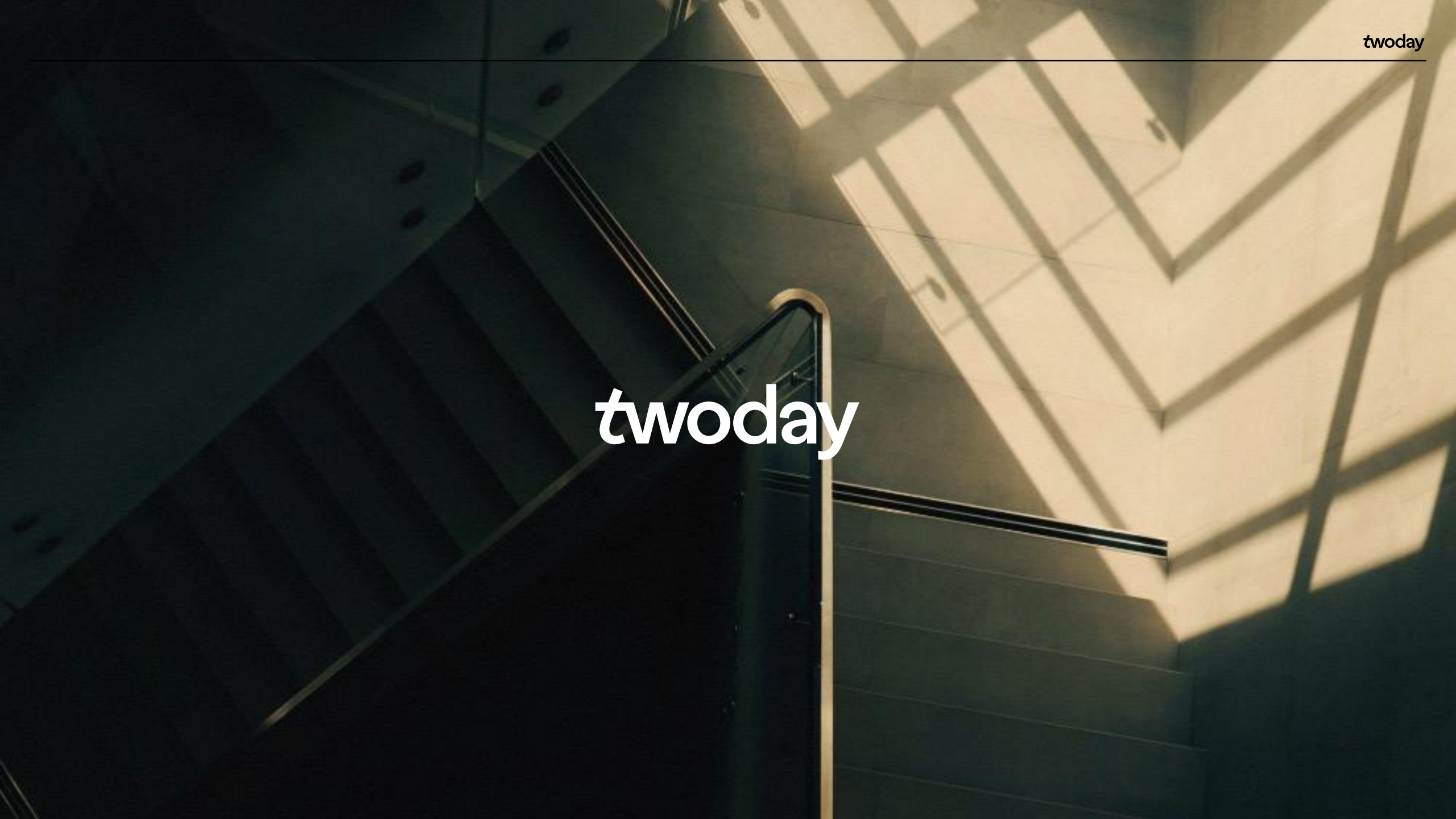


Prompt and context engineering

demo

Common mistakes

Overloading context	Duplicating instructions	Stale or conflicting context	Treating instructions as enforcement
Supplying too much material, making it harder to identify what's relevant.	Repeating architectural rules in every prompt instead of maintaining shared project guidance.	Supplying outdated documentation or instructions that disagree with the implementation.	Assuming that written rules guarantee compliance. Critical invariants still need tests, validation, and appropriate access controls.


twoday

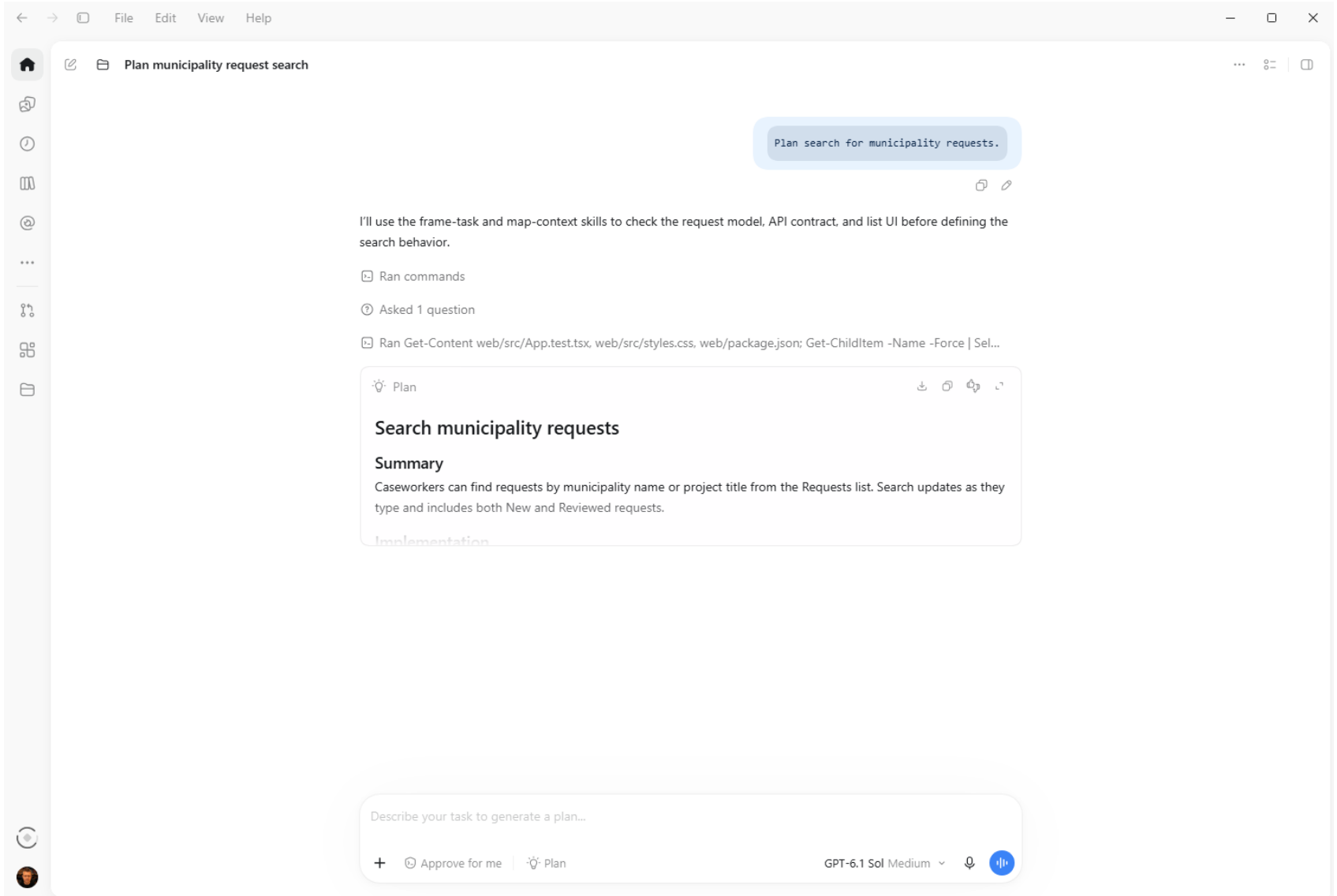
P1

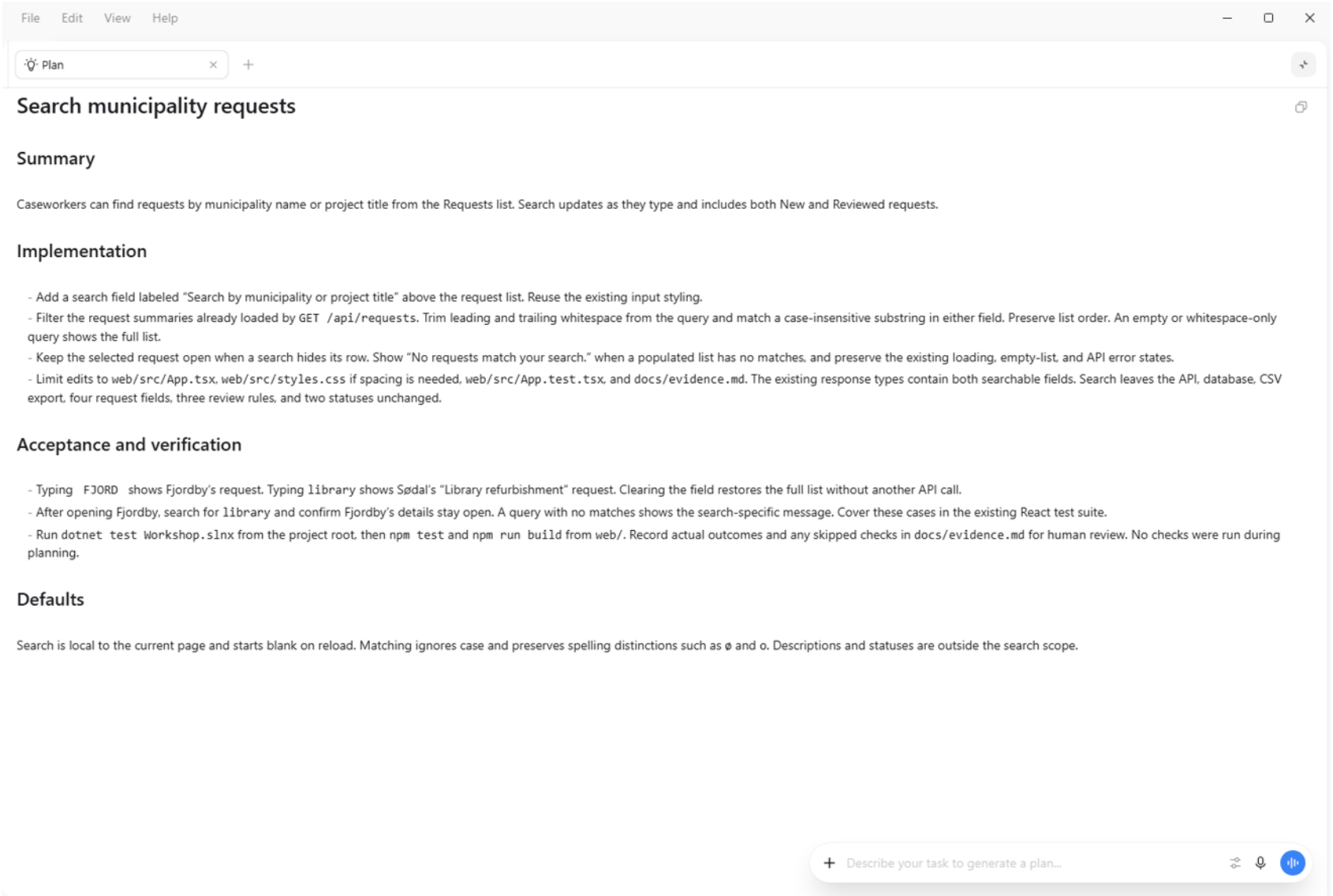
Prompt engineering

demo

Weak prompt

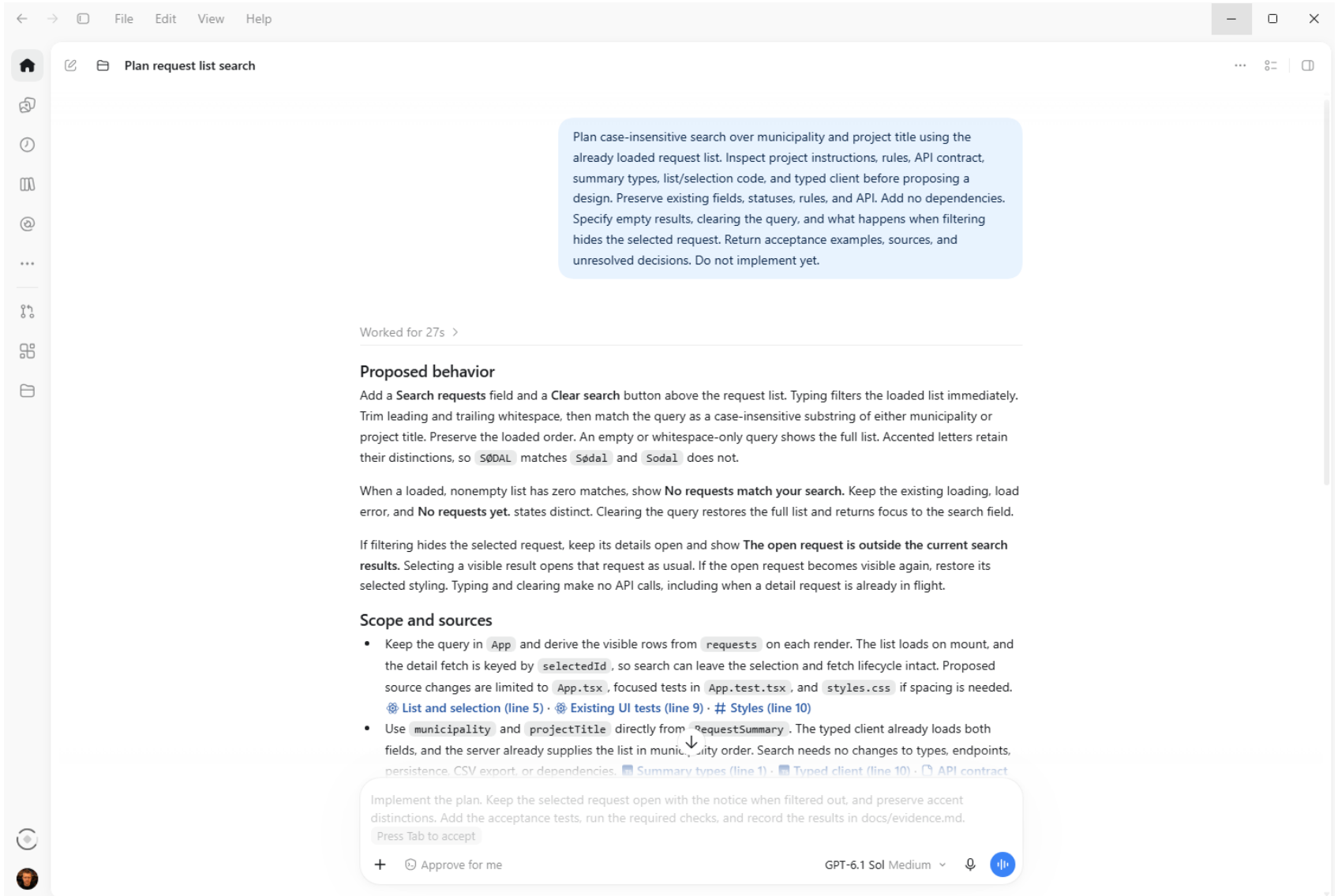
```
Plan search for municipality requests. Do not implement yet.
```

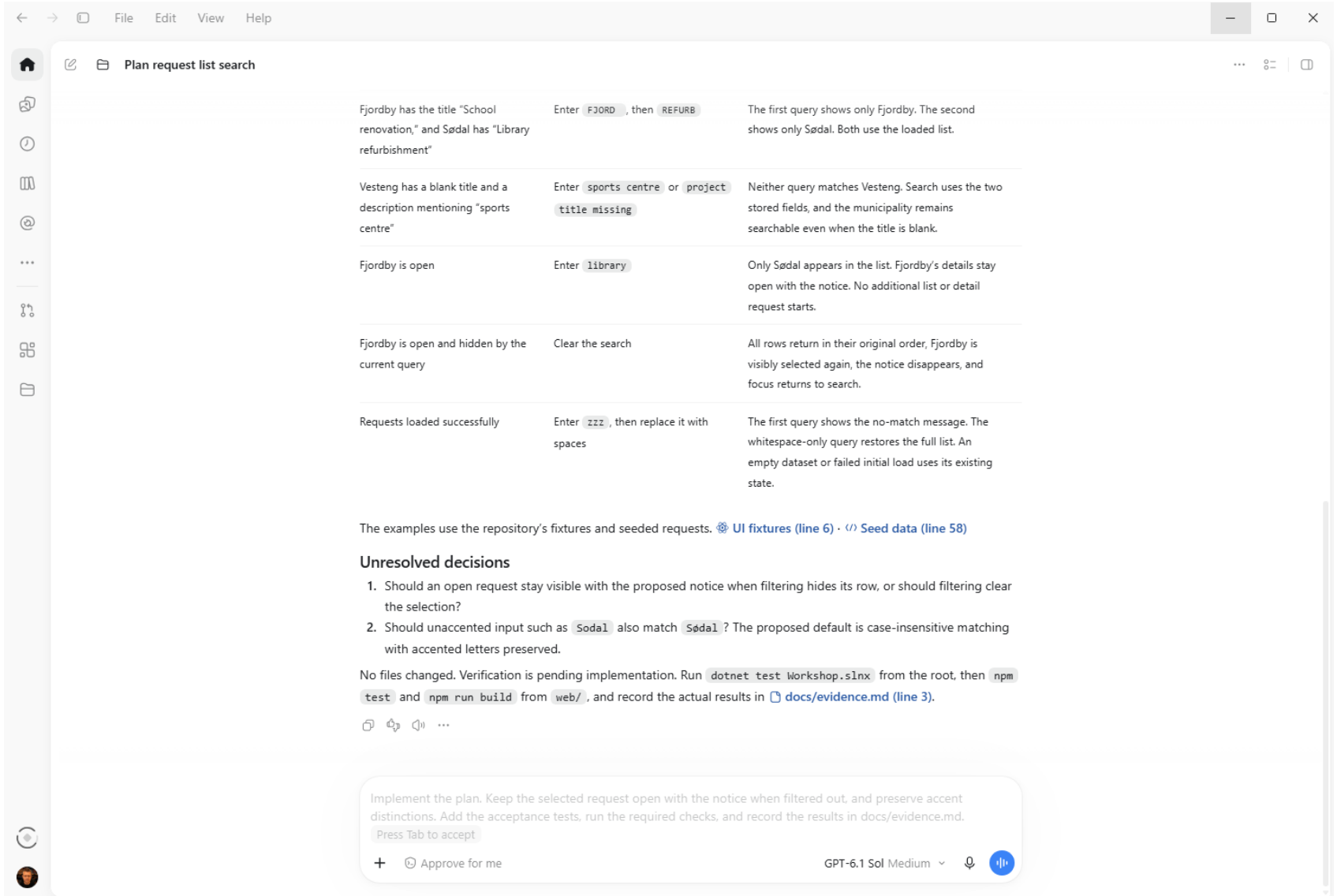

File
Edit
View
Help
Plan municipality request search
Plan search for municipality requests.
I'll use the frame-task and map-context skills to check the request model, API contract, and list UI before defining the search behavior.
Ran commands
Asked 1 question
Ran Get-Content web/src/App.test.tsx, web/src/styles.css, web/package.json; Get-ChildItem -Name -Force | Sel...
Plan
Search municipality requests
Summary
Caseworkers can find requests by municipality name or project title from the Requests list. Search updates as they type and includes both New and Reviewed requests.
Implementation
Describe your task to generate a plan...
Approve for me
Plan
GPT-6.1 Sol Medium

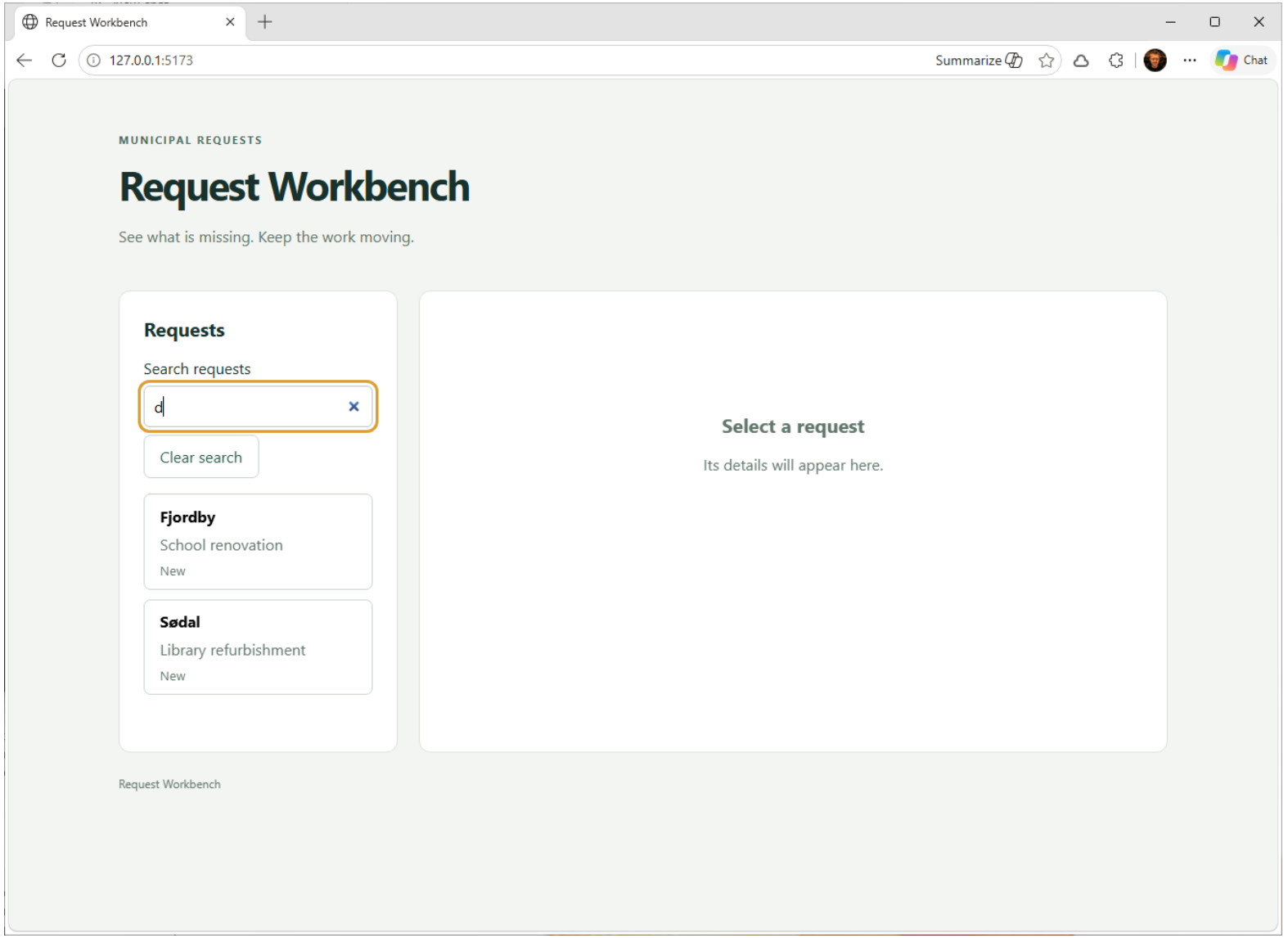

File
Edit
View
Help
Plan
Search municipality requests
Summary
Caseworkers can find requests by municipality name or project title from the Requests list. Search updates as they type and includes both New and Reviewed requests.
Implementation
- Add a search field labeled "Search by municipality or project title" above the request list. Reuse the existing input styling.
- Filter the request summaries already loaded by GET /api/requests. Trim leading and trailing whitespace from the query and match a case-insensitive substring in either field. Preserve list order. An empty or whitespace-only query shows the full list.
- Keep the selected request open when a search hides its row. Show "No requests match your search." when a populated list has no matches, and preserve the existing loading, empty-list, and API error states.
- Limit edits to web/src/App.tsx, web/src/styles.css if spacing is needed, web/src/App.test.tsx, and docs/evidence.md. The existing response types contain both searchable fields. Search leaves the API, database, CSV export, four request fields, three review rules, and two statuses unchanged.
Acceptance and verification
- Typing FJORD shows Fjordby's request. Typing library shows Sødal's "Library refurbishment" request. Clearing the field restores the full list without another API call.
- After opening Fjordby, search for library and confirm Fjordby's details stay open. A query with no matches shows the search-specific message. Cover these cases in the existing React test suite.
- Run dotnet test Workshop.slnx from the project root, then npm test and npm run build from web/. Record actual outcomes and any skipped checks in docs/evidence.md for human review. No checks were run during planning.
Defaults
Search is local to the current page and starts blank on reload. Matching ignores case and preserves spelling distinctions such as ø and o. Descriptions and statuses are outside the search scope.
Describe your task to generate a plan...

Engineered prompt

```
Plan case-insensitive search over municipality and project title using the
already loaded request list. Inspect project instructions, rules, API contract,
summary types, list/selection code, and typed client before proposing a design.
Preserve existing fields, statuses, rules, and API. Add no dependencies. Specify
empty results, clearing the query, and what happens when filtering hides the
selected request. Return acceptance examples, sources, and unresolved decisions.
Do not implement yet.
```


File
Edit
View
Help
Plan request list search
Plan case-insensitive search over municipality and project title using the already loaded request list. Inspect project instructions, rules, API contract, summary types, list/selection code, and typed client before proposing a design. Preserve existing fields, statuses, rules, and API. Add no dependencies. Specify empty results, clearing the query, and what happens when filtering hides the selected request. Return acceptance examples, sources, and unresolved decisions. Do not implement yet.
Worked for 27s
Proposed behavior
Add a Search requests field and a Clear search button above the request list. Typing filters the loaded list immediately. Trim leading and trailing whitespace, then match the query as a case-insensitive substring of either municipality or project title. Preserve the loaded order. An empty or whitespace-only query shows the full list. Accented letters retain their distinctions, so SØDAL matches Sødal and Sodal does not.
When a loaded, nonempty list has zero matches, show No requests match your search. Keep the existing loading, load error, and No requests yet. states distinct. Clearing the query restores the full list and returns focus to the search field.
If filtering hides the selected request, keep its details open and show The open request is outside the current search results. Selecting a visible result opens that request as usual. If the open request becomes visible again, restore its selected styling. Typing and clearing make no API calls, including when a detail request is already in flight.
Scope and sources
Keep the query in App and derive the visible rows from requests on each render. The list loads on mount, and the detail fetch is keyed by selectedId, so search can leave the selection and fetch lifecycle intact. Proposed source changes are limited to App.tsx, focused tests in App.test.tsx, and styles.css if spacing is needed.
List and selection (line 5) · Existing UI tests (line 9) · Styles (line 10)
Use municipality and projectTitle directly from RequestSummary. The typed client already loads both fields, and the server already supplies the list in muni ity order. Search needs no changes to types, endpoints, persistence, CSV export, or dependencies. Summary types (line 1) · Typed client (line 10) · API contract
Implement the plan. Keep the selected request open with the notice when filtered out, and preserve accent distinctions. Add the acceptance tests, run the required checks, and record the results in docs/evidence.md.
Press Tab to accept
Approve for me
GPT-6.1 Sol Medium


File
Edit
View
Help
Plan request list search
Fjordby has the title "School renovation," and Sødal has "Library refurbishment"
Enter FJORD, then REFURB
The first query shows only Fjordby. The second shows only Sødal. Both use the loaded list.
Vesteng has a blank title and a description mentioning "sports centre"
Enter sports centre or project title missing
Neither query matches Vesteng. Search uses the two stored fields, and the municipality remains searchable even when the title is blank.
Fjordby is open
Enter library
Only Sødal appears in the list. Fjordby's details stay open with the notice. No additional list or detail request starts.
Fjordby is open and hidden by the current query
Clear the search
All rows return in their original order, Fjordby is visibly selected again, the notice disappears, and focus returns to search.
Requests loaded successfully
Enter zzz, then replace it with spaces
The first query shows the no-match message. The whitespace-only query restores the full list. An empty dataset or failed initial load uses its existing state.
The examples use the repository's fixtures and seeded requests. UI fixtures (line 6) · Seed data (line 58)
Unresolved decisions
1. Should an open request stay visible with the proposed notice when filtering hides its row, or should filtering clear the selection?
2. Should unaccented input such as Sodal also match Sødal? The proposed default is case-insensitive matching with accented letters preserved.
No files changed. Verification is pending implementation. Run dotnet test Workshop.slnx from the root, then npm test and npm run build from web/, and record the actual results in docs/evidence.md (line 3).
Implement the plan. Keep the selected request open with the notice when filtered out, and preserve accent distinctions. Add the acceptance tests, run the required checks, and record the results in docs/evidence.md.
Press Tab to accept
Approve for me
GPT-6.1 Sol Medium


Request Workbench
127.0.0.1:5173
Summarize
Chat
MUNICIPAL REQUESTS
Request Workbench
See what is missing. Keep the work moving.
Requests
Search requests
d
Clear search
Fjordby
School renovation
New
Sødal
Library refurbishment
New
Select a request
Its details will appear here.
Request Workbench

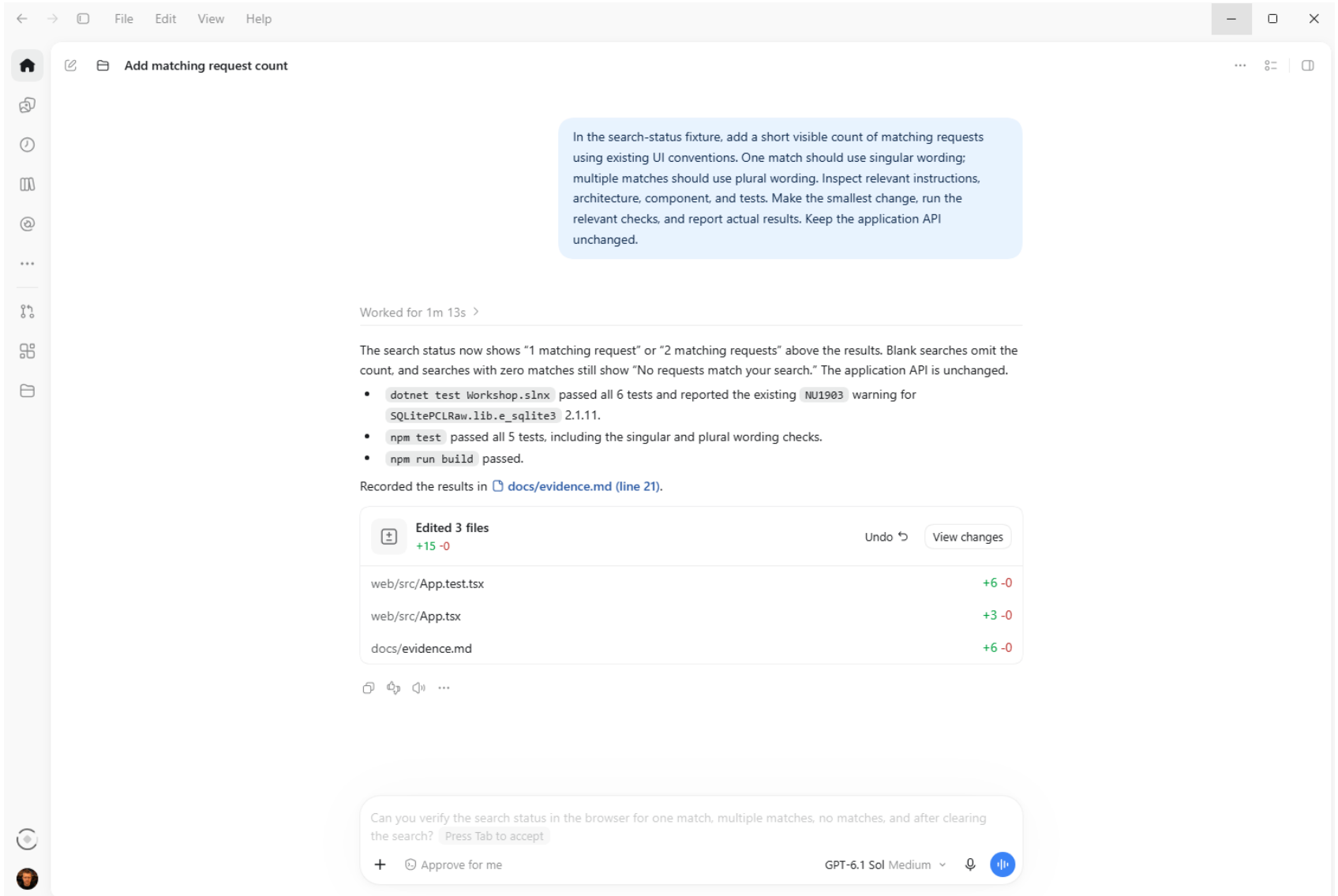
P2

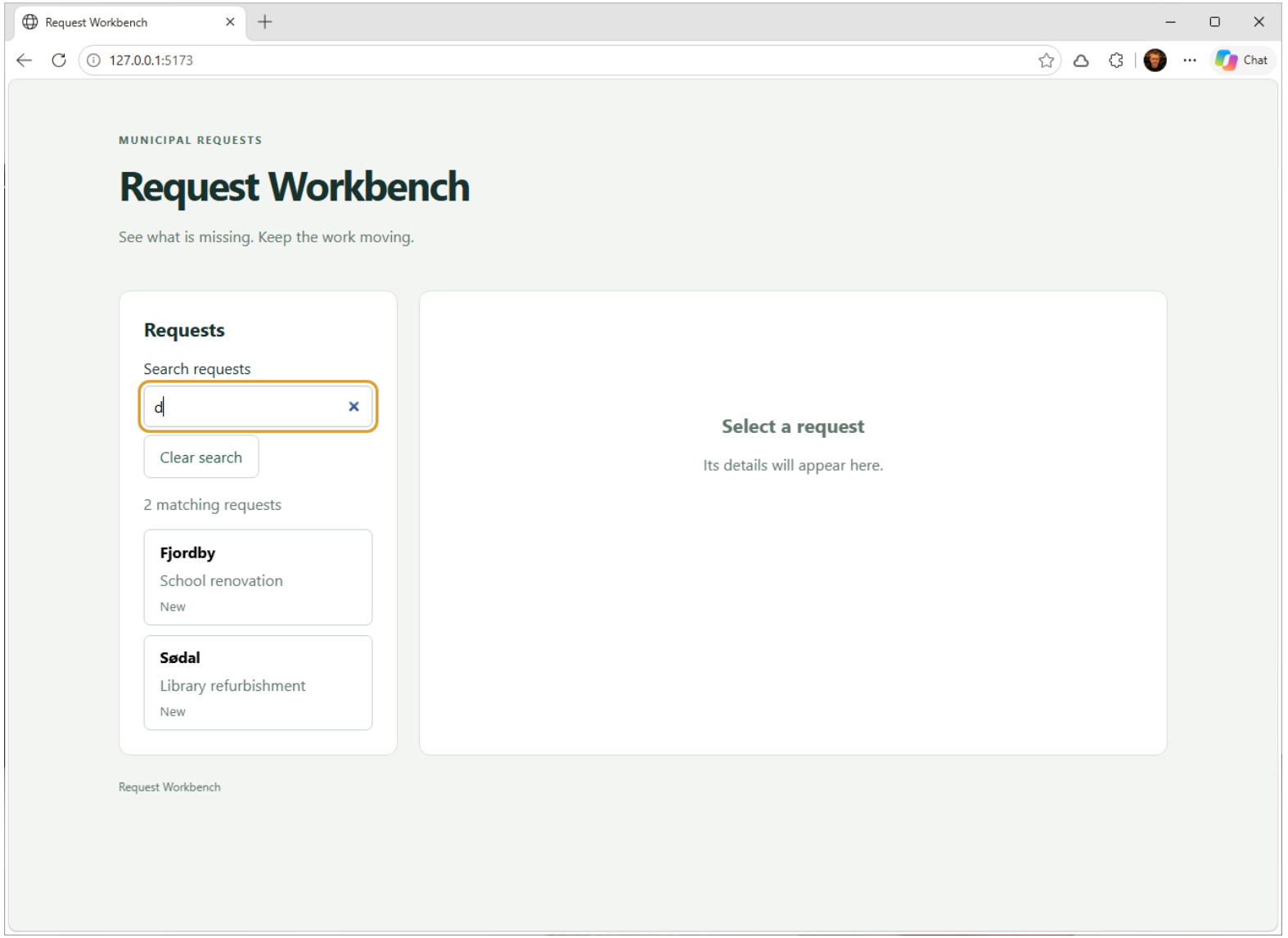
Context engineering

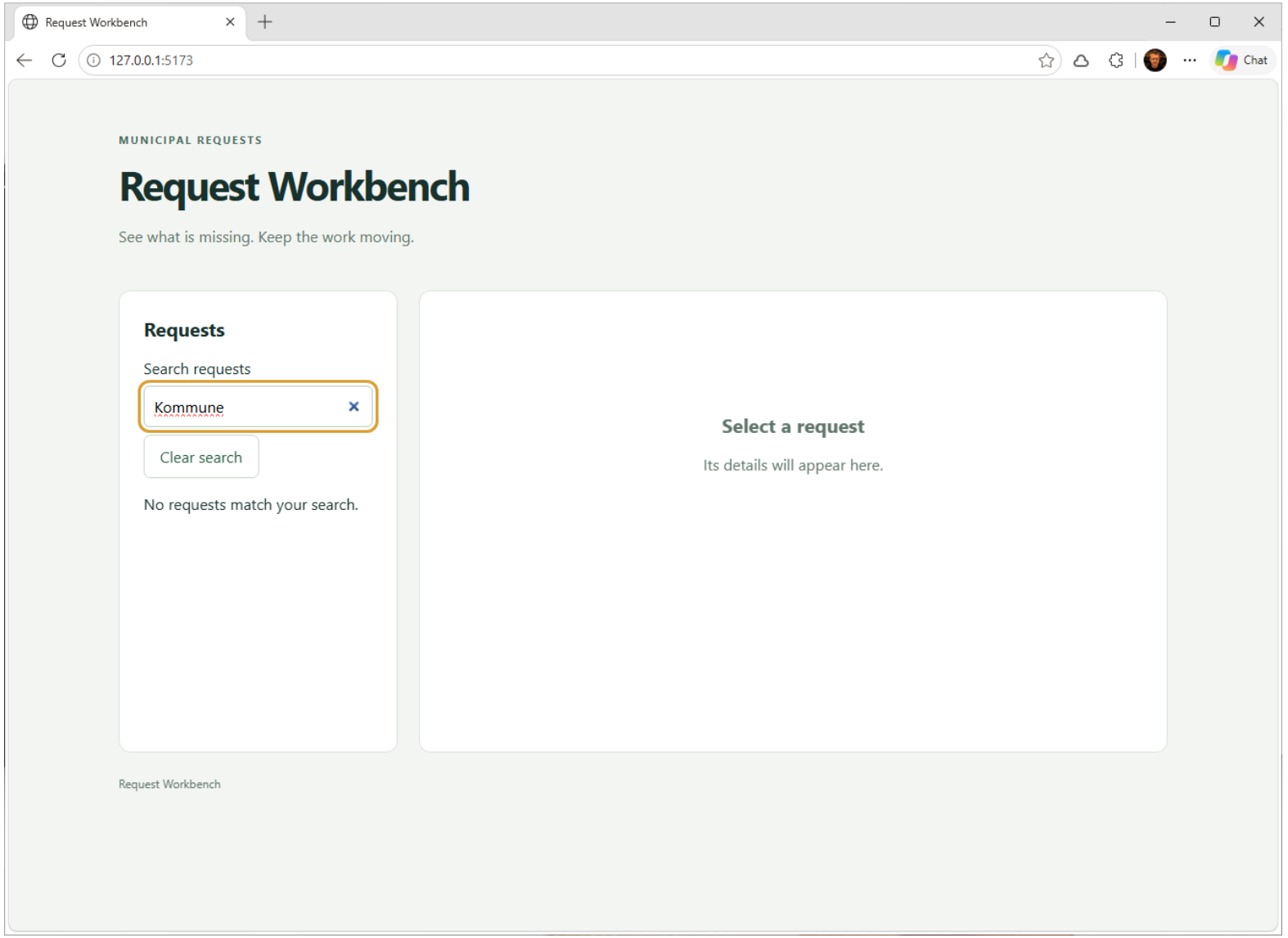
demo

Following context

In the search-status fixture, add a short visible count of matching requests using existing UI conventions. One match should use singular wording; multiple matches should use plural wording. Inspect relevant instructions, architecture, component, and tests. Make the smallest change, run the relevant checks, and report actual results. Keep the application API unchanged.

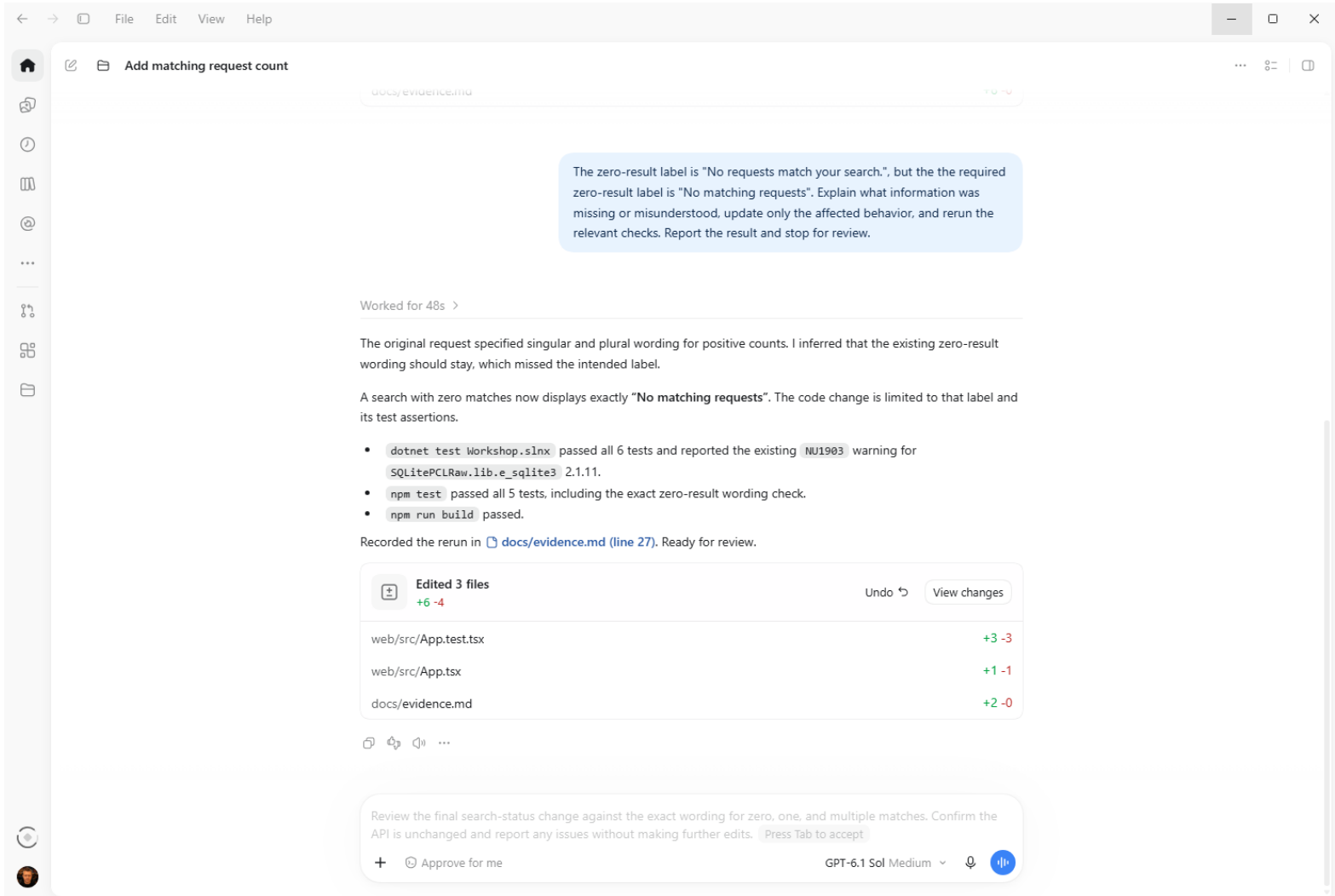

File
Edit
View
Help
Add matching request count
In the search-status fixture, add a short visible count of matching requests using existing UI conventions. One match should use singular wording; multiple matches should use plural wording. Inspect relevant instructions, architecture, component, and tests. Make the smallest change, run the relevant checks, and report actual results. Keep the application API unchanged.
Worked for 1m 13s
The search status now shows “1 matching request” or “2 matching requests” above the results. Blank searches omit the count, and searches with zero matches still show “No requests match your search.” The application API is unchanged.
dotnet test Workshop.slnx passed all 6 tests and reported the existing NU1903 warning for SQLitePCLRaw.lib.e_sqlite3 2.1.11.
npm test passed all 5 tests, including the singular and plural wording checks.
npm run build passed.
Recorded the results in docs/evidence.md (line 21).
Edited 3 files
+15 -0
Undo
View changes
web/src/App.test.tsx +6 -0
web/src/App.tsx +3 -0
docs/evidence.md +6 -0
Can you verify the search status in the browser for one match, multiple matches, no matches, and after clearing the search? Press Tab to accept
Approve for me
GPT-6.1 Sol Medium

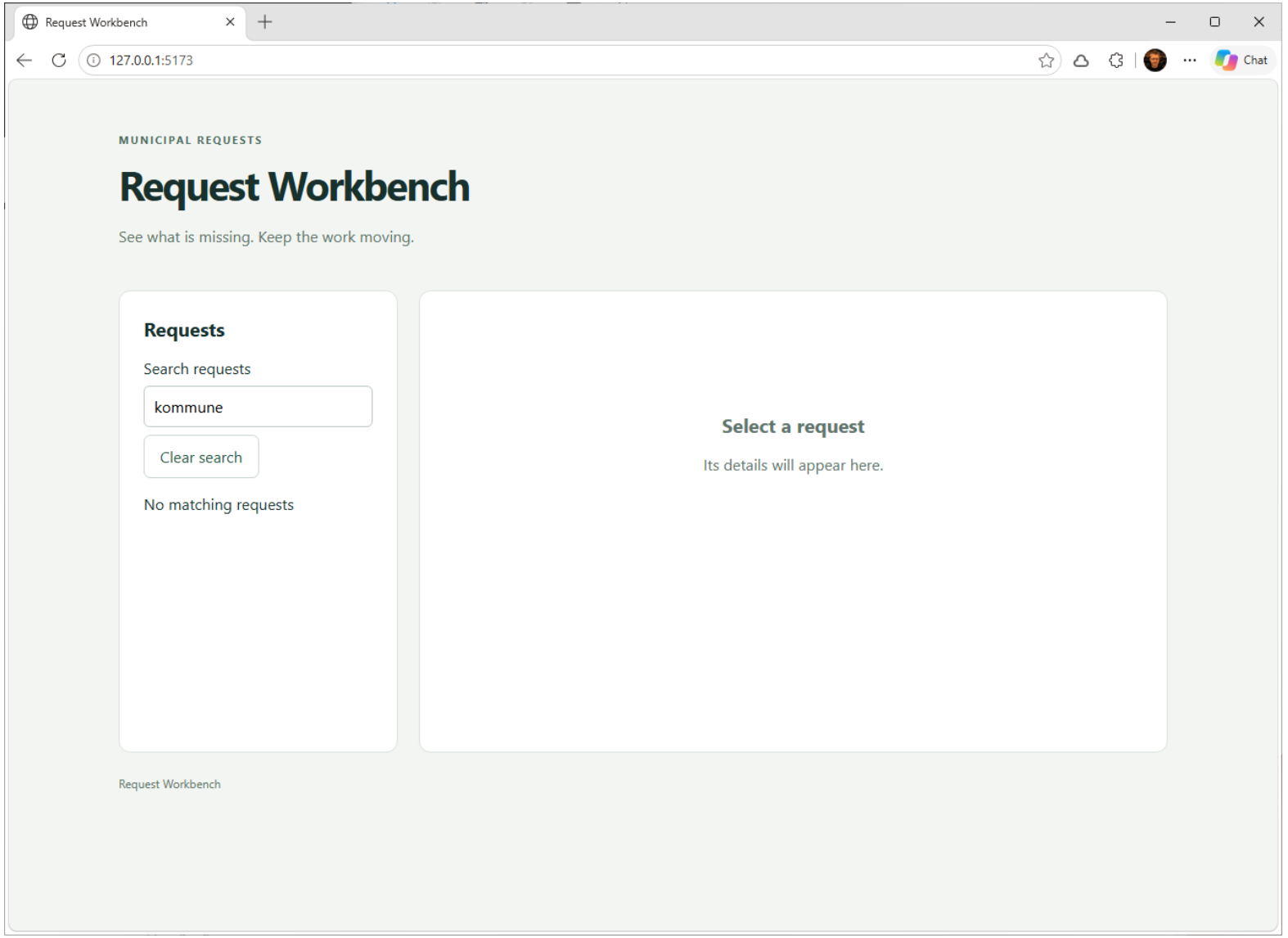

Request Workbench
127.0.0.1:5173
Chat
MUNICIPAL REQUESTS
Request Workbench
See what is missing. Keep the work moving.
Requests
Search requests
d
Clear search
2 matching requests
Fjordby
School renovation
New
Sødal
Library refurbishment
New
Select a request
Its details will appear here.
Request Workbench


Request Workbench
127.0.0.1:5173
Chat
MUNICIPAL REQUESTS
Request Workbench
See what is missing. Keep the work moving.
Requests
Search requests
Kommune
Clear search
No requests match your search.
Select a request
Its details will appear here.
Request Workbench

Follow up prompt

```
The required zero-result label is "No matching requests". Explain what
information was missing or misunderstood, update only the affected behavior, and
rerun the relevant checks. Report the result and stop for review.
```


File
Edit
View
Help
Add matching request count
The zero-result label is "No requests match your search.", but the the required zero-result label is "No matching requests". Explain what information was missing or misunderstood, update only the affected behavior, and rerun the relevant checks. Report the result and stop for review.
Worked for 48s
The original request specified singular and plural wording for positive counts. I inferred that the existing zero-result wording should stay, which missed the intended label.
A search with zero matches now displays exactly "No matching requests". The code change is limited to that label and its test assertions.
dotnet test Workshop.slnx passed all 6 tests and reported the existing NU1903 warning for SQLitePCLRaw.lib.e_sqlite3 2.1.11.
npm test passed all 5 tests, including the exact zero-result wording check.
npm run build passed.
Recorded the rerun in docs/evidence.md (line 27). Ready for review.
Edited 3 files
+6 -4
Undo
View changes
web/src/App.test.tsx +3 -3
web/src/App.tsx +1 -1
docs/evidence.md +2 -0
Review the final search-status change against the exact wording for zero, one, and multiple matches. Confirm the API is unchanged and report any issues without making further edits.
Press Tab to accept
Approve for me
GPT-6.1 Sol Medium


Request Workbench
127.0.0.1:5173
Chat
MUNICIPAL REQUESTS
Request Workbench
See what is missing. Keep the work moving.
Requests
Search requests
kommune
Clear search
No matching requests
Select a request
Its details will appear here.
Request Workbench